

Generic Delegates in C# with Real-Time Examples

In this article, I am going to discuss the **Generic Delegates in C#** with examples. Please read our previous article where we discussed [Delegate Real-Time Examples in C#](#) with Examples. As part of this article, we are going to discuss the following pointers in detail.

1. [What are Generic Delegates in C#?](#)
2. [Types of Generic Delegates in C#.](#)
3. [Why do we need the Generic Delegates?](#)
4. [Func Generic Delegate in C#?](#)
5. [Action Generic Delegate in C#?](#)
6. [Predicate Generic Delegate in C#?](#)
7. [Examples to understand Generic Delegates in C#.](#)

What are Generic Delegates in C#?

The Generic Delegates in C# were introduced as part of .NET Framework 3.5 which doesn't require defining the delegate instance in order to invoke the methods. To understand the Generic Delegates in C# you should have the basic knowledge of [Delegates](#).

Types of Generic Delegates in C#

C# provides three built-in generic delegates, they are as follows:

1. **Func**
2. **Action**
3. **Predicate**

Why do we need the Generic Delegates in C#?

Let us understand the need for Generic Delegates in C# with an example. In order to understand this, let us first understand how we use delegates to invoke methods. Let us say we have the following three methods and we want to invoke these methods using delegates.

```
public static double AddNumber1(int no1, float no2, double no3)
{
    return no1 + no2 + no3;
}

public static void AddNumber2(int no1, float no2, double no3)
{
    Console.WriteLine(no1 + no2 + no3);
}

public static bool CheckLength(string name)
{
    if (name.Length > 5)
        return true;
    return false;
}
```

As you can see in the above code, the **AddNumber1** method takes three parameters and returns a value of double type. Similarly, the **AddNumber2** method takes three parameters but it does not return any value and here the return type is void. The third method i.e. the **CheckLength** method takes one string parameter and returns a Boolean value. If the string length is greater than 5, then it will return true else it will return false.

Now, if we want to invoke the above three methods using delegates in C#, then we need to create three delegates whose signatures should match the signatures of the above three methods as shown in the below image.

```
public delegate double AddNumber1Delegate(int no1, float no2, double no3);
public delegate void AddNumber2Delegate(int no1, float no2, double no3);
public delegate bool CheckLengthDelegate(string name);
```

As you can see in the above image, we create three delegates. Now, once we have created the delegates. then we can invoke the methods by creating instances of each delegate referring to the respective methods and then we can invoke the delegate as shown in the below code.

```

AddNumber1Delegate obj1 = new AddNumber1Delegate(AddNumber1);
double Result = obj1.Invoke(100, 125.45f, 456.789);
Console.WriteLine(Result);

AddNumber2Delegate obj2 = new AddNumber2Delegate(AddNumber2);
obj2.Invoke(50, 255.45f, 123.456);

CheckLengthDelegate obj3 = new CheckLengthDelegate(CheckLength);
bool Status = obj3.Invoke("Pranaya");
Console.WriteLine(Status);

```

Example to Invoke Methods using Custom Delegate in C#:

Whatever we have discussed as of now, the complete code is given below.

```

using System;
namespace GenericDelegatesDemo
{
    public class GenericDelegates
    {
        public delegate double AddNumber1Delegate(int no1, float no2, double no3);
        public delegate void AddNumber2Delegate(int no1, float no2, double no3);
        public delegate bool CheckLengthDelegate(string name);
        static void Main(string[] args)
        {
            AddNumber1Delegate obj1 = new AddNumber1Delegate(AddNumber1);
            double Result = obj1.Invoke(100, 125.45f, 456.789);
            Console.WriteLine(Result);
            AddNumber2Delegate obj2 = new AddNumber2Delegate(AddNumber2);
            obj2.Invoke(50, 255.45f, 123.456);
            CheckLengthDelegate obj3 = new CheckLengthDelegate(CheckLength);
            bool Status = obj3.Invoke("Pranaya");
            Console.WriteLine(Status);
            Console.ReadKey();
        }
        public static double AddNumber1(int no1, float no2, double no3)
        {
            return no1 + no2 + no3;
        }
        public static void AddNumber2(int no1, float no2, double no3)
        {
            Console.WriteLine(no1 + no2 + no3);
        }
        public static bool CheckLength(string name)
        {

```

```

        if (name.Length > 5)
            return true;
        return false;
    }
}

```

Output:

```

682.238996948242
428.905996948242
True

```

As of now, this is the way, we use delegates to invoke methods. The question that comes to our mind is do we really need to create custom Delegates to invoke the methods in C#?

Do we really need to create Custom Delegates to Invoke Methods in C#?

The answer is no. C#.NET Framework provides some generic delegates who can do the job for us. C# provides three Generic Delegates; they are as follows

1. **Func**
2. **Action**
3. **Predicate**

Now, let us proceed and try to understand all the above three generic delegates. Let us try to understand what are all these delegates, when, and how to use all these generic delegates in C# with examples.

What is Func Generic Delegate in C#?

The Func Generic Delegate in C# is present in the System namespace. This delegate takes one or more input parameters and returns one out parameter. The last parameter is considered as the return value. The Func Generic Delegate in C# can take up to 16 input parameters of different or the same data types. It must have one return type. The return type is mandatory but the input parameter is not mandatory.

Note: Whenever your delegate returns some value, whether by taking any input parameter or not, you need to use the Func Generic delegate in C#.

What is Action Generic Delegate in C#?

The Action Generic Delegate in C# is also present in the System namespace. It takes one or more input parameters and returns nothing. This delegate can take a maximum of 16 input parameters of the different or same data types.

Note: Whenever your delegate does not return any value, whether by taking any input parameter or not, then you need to use the Action Generic delegate in C#.

What is Predicate Generic Delegate in C#?

The Predicate Generic Delegate in C# is also present in the System namespace. This delegate is used to verify certain criteria of the method and returns the output as Boolean, either True or False. It takes one input parameter and always returns a Boolean value which is mandatory. This delegate can take a maximum of 1 input parameter and always return the value of the Boolean type.

Note: Whenever your delegate returns a Boolean value, by taking only one input parameter, then you need to use the Predicate Generic delegate in C#.

Examples to understand Generic Delegates in C#.

Let us understand the above three generic delegates in C# with an example. In our first example, we created three methods,

1. The **AddNumber1** method takes three parameters and returns a **double** value. Here we will use the **Func Generic Delegate** to achieve the same thing as we achieve in the first example.
2. Similarly, the **AddNumber2** method takes three parameters but does not return any value. Here we will use the **Action Generic Delegate** to achieve the same thing as we achieve in the first example.
3. The **CheckLength** method takes one string input parameter and returns a boolean value. Here we will use the **Predicate Generic Delegate** to achieve the same thing as we achieve in the first example.

Let us first remove the three Delegates that we created in our application and see how can we convert the same application using the generic delegates.

How to use Func Generic Delegate in C#?

The Func Generic Delegate in C# is used whenever your delegate returns some value, whether by taking any input parameter or not. In our example, the AddNumber1 method takes some input and returns one output. So, the AddNumber1 method signature is matched with the Func generic delegate signature. So, here, instead of creating our own delegate to invoke the AddNumber1 method, we can use the Func generic delegate to invoke the AddNumber1 method as shown in the below code.

```
AddNumber1Delegate obj1 = new AddNumber1Delegate(AddNumber1);  
double Result = obj1.Invoke(100, 125.45f, 456.789);  
Console.WriteLine(Result);
```

← Using Custom Delegate

Input Parameters

Changes to

```
Func<int, float, double, double> obj1 = new Func<int, float, double, double>(AddNumber1);  
double Result = obj1.Invoke(100, 125.45f, 456.789);  
Console.WriteLine(Result);
```

Output Parameter

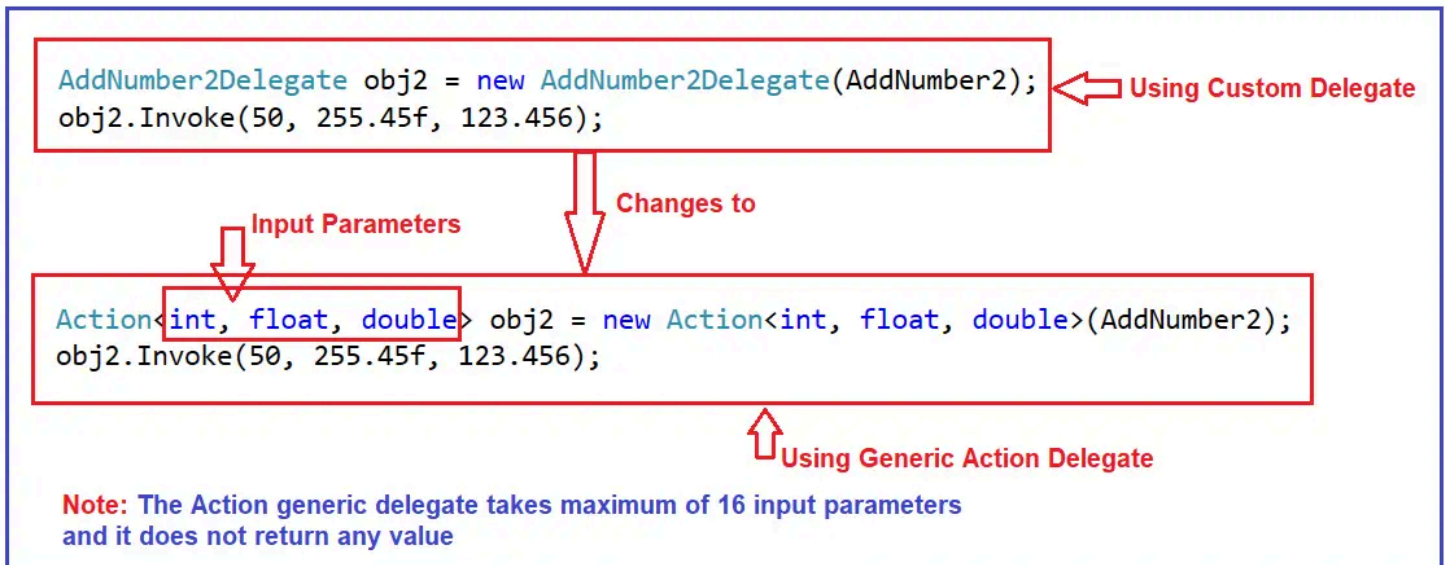
Using Generic Delegate

Note: Always the last parameter in the parameter list is the output parameter or the return value type in the Func Generic Delegate in C#

As shown in the above code, the Func Generic Delegate takes four parameters, the first three are input parameters and the last one is the return value. To the Func Generic Delegate constructor, we pass the AddNumber1 method which is going to execute when we invoke the Func delegate.

How to use Action Generic Delegate in C#?

The Action Generic Delegate in C# is used whenever your delegate does not return any value, whether by taking any input parameter or not. In our example, the AddNumber2 method takes some input but does not return any value. So, the AddNumber2 method signature is matched with the Action generic delegate signature. So, here, instead of creating our own delegate to invoke the AddNumber2 method, we can use the Action generic delegate to invoke the AddNumber2 method as shown in the below code.



As shown in the above code, the Action Generic Delegate takes three input parameters. To the Action Generic Delegate constructor, we pass the AddNumber2 method which is going to execute when we invoke the Action delegate. The Action Generic Delegate never going to return any value.

How to use Predicate Generic Delegate in C#?

The Predicate Generic Delegate in C# is used whenever your delegate returns a Boolean value, by taking only one input parameter. In our example, the CheckLength method takes one input parameter of string type and returns a Boolean value. So, the CheckLength method signature is matched with the Predicate generic delegate signature. So, here, instead of creating our own delegate to invoke the CheckLength method, we can use the Predicate generic delegate to invoke the CheckLength method as shown in the below code.

```

CheckLengthDelegate obj3 = new CheckLengthDelegate(CheckLength);
bool Status = obj3.Invoke("Pranaya");
Console.WriteLine(Status);

```



Using Custom Delegate



Changes to



Only one Input Parameter

```

Predicate<string> obj3 = new Predicate<string>(CheckLength);
bool Status = obj3.Invoke("Pranaya");
Console.WriteLine(Status);

```



Using Generic Predicate Delegate

Note: The Predicate Delegate takes maximum of 1 Input Parameter of any type, and returns a boolean value. We don't have to specify the return value as by Default the return value type is boolean.

As shown in the above code, the Predicate Generic Delegate takes one string input parameter. To the Predicate Generic Delegate constructor, we pass the CheckLength method which is going to execute when we invoke the Predicate Generic delegate. This delegate can take a maximum of 1 input parameter and 0 return values. By default, it returns a Boolean value.

Example to Understand Func, Action, and Predicate Generic Delegates in C#:

Whatever we have discussed, let us put all these things into our application. In the following example, we are using the three generic delegates instead of our own custom delegate to invoke the three methods.

```

using System;
namespace GenericDelegateDemo
{
    public class GenericDelegates
    {
        static void Main(string[] args)
        {
            Func<int, float, double, double> obj1 = new Func<int, float, double, double>(AddNumber1);
            double Result = obj1.Invoke(100, 125.45f, 456.789);
            Console.WriteLine(Result);
            Action<int, float, double> obj2 = new Action<int, float, double>(AddNumber2);
            obj2.Invoke(50, 255.45f, 123.456);
            Predicate<string> obj3 = new Predicate<string>(CheckLength);
            bool Status = obj3.Invoke("Pranaya");
            Console.WriteLine(Status);
            Console.ReadKey();
        }
        public static double AddNumber1(int no1, float no2, double no3)
        {
            return no1 + no2 + no3;
        }
        public static void AddNumber2(int no1, float no2, double no3)
        {

```

```

        Console.WriteLine(no1 + no2 + no3);
    }
    public static bool CheckLength(string name)
    {
        if (name.Length > 5)
            return true;
        return false;
    }
}

```

Output:

```

682.238996948242
428.905996948242
True

```

Generic Delegates with Lambda Expression in C#:

Let us see how to use Lambda Expression along with Generic Delegates in C#. The following is the same example as the previous example. But in this case, while creating the instance of generic delegates, instead of the methods, we are using lambda expressions and when we invoke the delegate, the respective lambda expression is going to be executed.

```

using System;
namespace GenericDelegateDemo
{
    public class GenericDelegates
    {
        static void Main(string[] args)
        {
            Func<int, float, double, double> obj1 = (x, y, z) =>
            {
                return x + y + z;
            };
            double Result = obj1.Invoke(100, 125.45f, 456.789);
            Console.WriteLine(Result);
            Action<int, float, double> obj2 = (x, y, z) =>
            {
                Console.WriteLine(x + y + z);
            };
            obj2.Invoke(50, 255.45f, 123.456);
            Predicate<string> obj3 = new Predicate<string>(CheckLength);
            bool Status = obj3.Invoke("Pranaya");
            Console.WriteLine(Status);
            Console.ReadKey();
        }
        public static bool CheckLength(string name)
        {

```

```
        if (name.Length > 5)
            return true;
        return false;
    }
}
```

Output:

```
682.238996948242
428.905996948242
True
```

Points to Remember while working with C# Generic Delegates:

1. Func, Action, and Predicate are Generic Inbuilt delegates that are present in the System namespace which is introduced in C# 3.
2. All these three delegates can be used with the method, [Anonymous Method](#), and [Lambda Expressions](#) in C#.
3. The Func delegates can contain a maximum of 16 input parameters and must have one return type and that will be the last parameter in the parameter list.
4. Action delegate can contain a maximum of 16 input parameters and does not have any return type.
5. The Predicate delegate should satisfy some criteria of a method and must have only one input parameter. By default, it is having one output parameter of return type and we don't have to pass the output parameter to the Predicate.

In the next article, I am going to discuss [Anonymous Method in C#](#) with Examples. Here, in this article, I try to explain **Generic Delegates in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.